CSE142 Object Oriented Programming Techniques
LAB- EXAM Spring'25

Time Allowed: 120 minutes
Max Marks: 45
Student ID: ————————-

Instructions

- Cell phones are not allowed. No internet or helping material allowed.
- You have to submit your exam on LMS. Only .cpp files zipped in a folder (folder name must be your ERP).
- You must write ERP on this question paper.
- Understanding the question is part of exam, You are not allowed to ask any question from your instructor.
- Attempt any three questions. All questions carry equal marks.

Exercise 1 .....

Designing Vehicle Management System

You are developing a lightweight Vehicle Management System using object-oriented programming in C++. Your implementation must demonstrate the use of *multi-level inheritance*, *virtual functions*, *operator overloading*, and *return-value usage*.

Complete the following tasks:

1. Create an abstract base class `Vehicle` with:
   - Protected member: `std::string brand`.
   - Pure virtual functions:
     - `double topSpeed() const`
     - `void display() const`
   - A virtual destructor.
2. Create an abstract intermediate class `MotorVehicle` inheriting from `Vehicle`, with:
   - Protected member: `bool electric`.
   - Additional pure virtual function: `bool isElectric() const`
3. Implement a concrete class `Car` inheriting from `MotorVehicle`, with:
   - Private member: `double enginePower`.
   - `topSpeed()` returns `enginePower * 2.5`.
   - `isElectric()` returns the value of `electric`.
   - `display()` prints the brand, power, and electric status.
4. Overload:
   - `+` to return the sum of two cars' top speeds.
   - `«` to print car details using `display()`.
5. In your `main()` function:

- Dynamically create two `Car` objects via base class pointers.
- Print their details using «.
- Use `dynamic_cast` to check if each base-class pointer actually points to a `MotorVehicle`. If so, call `isElectric()` and print a message such as "This car is electric" or "This car is not electric".
- Compute and print the sum of their top speeds using `+`.
- Properly deallocate memory.

Note: Focus on demonstrating correct use of inheritance, virtual functions, operator overloading, and dynamic memory handling.

Exercise 2

Singly Linked List – Implementation and Reordering

You are given the following initial singly linked list created by inserting nodes at the front using `pushFront()`:

$$\boxed{50} \rightarrow \boxed{40} \rightarrow \boxed{30} \rightarrow \boxed{20} \rightarrow \boxed{10} \rightarrow \texttt{NULL}$$

Your task is to implement a **Singly Linked List** in C++ that supports basic insertion and deletion operations, and includes a reordering function based on node positions.

Part I: Linked List Class Implementation

Define a class `SinglyLinkedList` that contains:

- A private `Node* head` pointer.
- A `Node` struct with:
  - An integer value
  - A pointer to the next node

Implement the following public methods:

- `void pushFront(int value)` – Insert a node at the beginning of the list.
- `int popFront()` – Remove and return the value from the front. Throw `std::out_of_range` if the list is empty.
- `void insertAt(int value, int index)` – Insert a node at the given index (0-based). Throw an exception if index is invalid.
- `void display()` – Print the list from head to tail.

Part II: Odd-Even Reordering

Write a function:

```
Node* reorderOddEven(Node* head);
```

This function should rearrange the list so that:

- All nodes at **odd indices (1-based)** appear before nodes at **even indices**.
- The relative order in each group (odd and even) is preserved.
- The reordering is done **in-place** with $O(1)$ space and $O(n)$ time.

Example

Original list:

$$\boxed{50}_1 \rightarrow \boxed{40}_2 \rightarrow \boxed{30}_3 \rightarrow \boxed{20}_4 \rightarrow \boxed{10}_5 \rightarrow \texttt{NULL}$$

After calling `reorderOddEven(head);`:

$$\boxed{50}_1 \rightarrow \boxed{30}_3 \rightarrow \boxed{10}_5 \rightarrow \boxed{40}_2 \rightarrow \boxed{20}_4 \rightarrow \texttt{NULL}$$

Part III: Main Function Requirements

In your `main()` function:

- Create the list by inserting 10, 20, 30, 40, 50 using `pushFront()`.
- Print the list using `display()`.
- Call `reorderOddEven()` and update the head pointer.
- Print the modified list.

Note: Focus on pointer manipulation and efficient in-place reordering logic.

Exercise 3 .....

Shell Sort

Implement a **Shell Sort** for a vector of strings.

- Use a suitable **gap sequence** (e.g., Shell's original, Knuth's).
- Sort **in-place** using **shifting**, not swapping.
- Accept an optional custom comparator (e.g., case-insensitive).

Example

Input: `{"Apple", "banana", "Cherry", "date"}`

- Ascending: `{"Apple", "Cherry", "banana", "date"}`
- Case-insensitive: `{"Apple", "banana", "Cherry", "date"}`

Constraints:

- Maximum 1000 elements.
- Do not use built-in sorting functions.

Exercise 4 .....

Safe Robot Deployment in Factory Grid (Recursive Backtracking)

You are managing an $N \times N$ factory. Place K robots with these constraints:

- No robots adjacent **horizontally or vertically**.

Count Valid Robot Placements

Given an $N \times N$ grid, count the number of ways to place exactly K robots such that:

- No two robots are adjacent (vertically, horizontally, or diagonally).

Function Signature

```
int countRobotPlacements(std::vector<std::vector<int»& grid, int N, int K, int row);
```

- Use **recursive backtracking** to try placing robots row by row.
- At each step, validate that the current placement does not violate:
  - Row uniqueness (one robot per row),
  - Column uniqueness,
  - Adjacency constraints (no adjacent robots).
- Count and return the number of valid configurations.

Example

- Input: `N = 4, K = 2`
- Output: `78` (number of valid placements of 2 non-adjacent robots on a 4x4 grid)